Name :- Aaryan V. Rodrane Roll No :- 5004 Div :- 5
Batch :- A1 Branch :- AI&DS SAP ID :- 60018250070

Experiment No 11 & 12

Title :- Object - oriented Programming and inheritance, overriding and polymorphism

Aim :- To Implement the Object Oriented Programming concept in Python, write Python programs to implement Inheritance & Polymorphism.

Theory :-

Q1 What is Object - Oriented Programming (OOP)? How is it different from procedural programming?

=> OOP is a style of programming where a program is built from objects that hold data and the methods that work on the data. Its main ideas are encapsulation, inheritance and polymorphism.

Basis	Procedural Programming	Object oriented Programming
Approach	Program is a sequence of functions	Program is a set of interacting objects
Data	Shared openly between functions	Kept inside objects with their methods.
Reuse	Limited, by calling functions	High, through inheritance.

Sundaram
FOR EDUCATIONAL USE

Q2 Define the following terms with examples
a) class
b) object
c) constructor
d) destructor

=>a) class:-

A class is a blueprint that defines the attributes and methods of a group of similar objects.

b) Object:-

An object is an instance of a class. Each object holds its own copy of the data.

c) constructor:-

A constructor is the special method that runs automatically when an object is created and sets its initial values.

d) Destructor:-

A Destructor is the special method that runs when an object is destroyed, and it is used to release resources.

Sundaram
FOR EDUCATIONAL USE

code :-

```
class student:
    college = "DJSCE"

    def __init__(self, name, roll):
        self.name = name
        self.roll = roll
        print("Object created for", self.name)

    def show(self):
        print("Name:", self.name, "| Roll:",
              self.roll)
    def __del__(self):
        print("Object destroyed for", self.name)

s1 = student("Rahul", 12)
s1.show()
del s1
```

Output :-

```
Object created for Rahul
Name: Rahul | Roll: 12
Object destroyed for Rahul
```

Sundaram
FOR EDUCATIONAL USE

Q3 what is the purpose of the __init__() method in Python classes?

⇒ The init method is the constructor of a class. It runs automatically at object creation and assigns starting values to the attributes.

code:

```
class Book:
    def __init__(self, title, price):
        self.title = title   # attribute set at creation
        self.price = price

b = Book("Python Basics", 350)
print(b.title, b.price)
```

output:

Python Basics 350

Q4 what do you mean by inheritance? List and Explain any two types of inheritance supported in Python.

⇒ Inheritance lets a child class acquire the attributes and methods of a parent class, which avoids rewriting code. Single inheritance means one child derives from one parent. Multilevel inheritance means a class derives from a child class, forming a chain.

Sundaram
FOR EDUCATIONAL USE

code:

```
# Single inheritance
class Vehicle:
    def start(self):
        print("Vehicle started")

class Car(Vehicle):
    def drive(self):
        print("Car is moving")

c = Car()
c.start()
c.drive()

# Multilevel inheritance
class Animal:
    def eat(self):
        print("Animal eats")

class Dog(Animal):
    def bark(self):
        print("Dog barks")

class Puppy(Dog):
    def sleep(self):
        print("Puppy sleeps")
p = Puppy()
p.eat()
p.bark()
p.sleep()
```

Sundaram FOR EDUCATIONAL USE

output :-

Vehicle started
car is moving
Animal eats
Dog barks
Puppy weeps

Q5 what is method overriding? How does it differ from method overloading?

⇒ Method overriding means a child class gives its own version of a method already defined in the parent. Overloading means several methods share one name with different parameters.

Basis	Overriding	Overloading
Where	Between parent and child class	Within the same class.
Signature	Same name and parameters	Same name, different parameters.
Python	Supported directly	Not supported directly, default arguments are used.

Sundaram
FOR EDUCATIONAL USE

code:

```
class Shape:
    def area(self):
        print("area of a generic shape")

class Square(Shape):

    def __init__(self, side):
        self.side = side
    def area(self):
        print("area of square:", self.side *
                                 self.side)

Square(4).area()
Shape().area()

# overloading style: One method, default arguments
class calc:
    def add(self, a, b=0, c=0):
        return (a+b+c)

k = calc()
print(k.add(2))
print(k.add(2,3))
print(k.add(2,3,4))
```

Output:
```
area of square: 16
area of a generic shape
2
5
9
```

Sundaram FOR EDUCATIONAL USE

Q6 Explain the use of the super() function with an example.

⇒ The super() function give acess to the parent class, so the child can call the parent constructor or methods without naming the parent

Code:-

```
class Person:
    def __init__(self, name):
        self.name = name

class Employee(Person):
    def __init__(self, name, salary):
        super().__init__(name)
        self.salary = salary

    def display(self):
        print("Name:", self.name)
        print("Salary:", self.salary)

e = Employee("Riya", 45000)
e.display()
```

Output:-

```
Name: Riya
Salary: 45000
```

Sundaram
FOR EDUCATIONAL USE

Q7 What is polymorphism? Provide an example demonstrating polymorphism in Python.

⇒ Polymorphism means one method name behaving differently for different objects. In the example, each class defines sound() in its own way and the same call works for all

code:

```
class Cat:
    def sound(self):
        return "Meow"
class Cow:
    def sound(self):
        return "Moo"
class Duck:
    def sound(self):
        return "Quack"

for animal in (Cat(), Cow(), Duck()):
    print(animal.sound())
print(len("Python"), len([1,2,3]), len({"a":1}))
```

Output:

```
Meow
Cow
Quack
6 3 1
```

Sundaram
FOR EDUCATIONAL USE

Q8 what is the role of the self parameter in Python class methods?

⇒ The self parameter refers to the current object. It is the first parameter of every instance method and is used to reach the attributes and methods of that object. Python passes it automatically.

code:

```
class Counter:
    def __init__(self, start):
        self.value = start
    def increase(self):
        self.value += 1
        return self
c = Counter(5)
c.increase().increase()
print("Value:", c.value)
```

Output:

Value: 7

Sundaram
FOR EDUCATIONAL USE

@9 write a short note on the advantage of using OOP principles in real-world applications.

⇒ OOP makes real world applications easier to build and maintain. The main advantages are given below.

Advantage	Explanation	Real world use
Reusability	Inheritance lets existing code be reused	Savings and current accounts reuse a common account class
Easy maintenance	Code is divided into classes, so changes stay local	fixing the student class does not disturb other classes.
Data Protection	Encapsulation keeps data inside the object	Account balance is changed only through its methods
Real entity modelling	Objects map naturally to real things	Students, vehicles and Employees.
Flexibility	Polymorphism lets one call work on many objects.	Different shapes use one area() call
Scalability	New classes can be added without rewriting old ones	Adding a new product prototype in a shop system.

Sundaram
FOR EDUCATIONAL USE

Programs:

Program 1: Inheritance (single and multilevel)

```
# Base class
class Employee:
    def __init__(self, name, empId):
        self.name = name
        self.empId = empId
    def details(self):
        print("Name:", self.name)
        print("ID:", self.empID)
# child of Employee (single inheritance)
class Manager(Employee):
    def __init__(self, name, empId, department):
        super().__init__(name, empId)
        self.department = department
    def details(self):
        super().details()
        print("Department:", self.department)
# child of Manager (multilevel inheritance)
class Manager(Employee):
    def __init__(self, name, empId, department, company):
        super().__init__(name, empId, department)
        self.company = company
    def details(self):
        super().details()
        print("company:", self.company)
```

Sundaram
FOR EDUCATIONAL USE

```
m = Manager("Neha", 101, "Sales")
m.details()
print()
d = Director("Arjun", 1, "Finance", "TechCorp")
d.details()
```

Output:

```
Name: Neha
ID: 101
Department: Sales

Name: Arjun
ID: 1
Department: Finance
Company: TechCorp
```

Program 2: Polymorphism using method overriding

```
import math
class Shape:
    def area(self):
        return 0
class Rectangle(Shape):
    def __init__(self, length, breadth):
        self.length = length
        self.breadth = breadth
```

Sundaram
FOR EDUCATIONAL USE

```
    def area(self)
        return self.length * self.breadth

class circle(shape):
    def __init__(self, radius)
        self.radius = radius
    def area(self):
        return round(math.pi * self.radius **2 , 2)

class Triangle(shape):
    def __init__(self, base, height):
        self.base = base
        self.height = height
    def area(self):
        return 0.5 * self.base * self.height

shapes = [Rectangle(5,3), circle(7) Triangle(6, 4)]
for s in shapes:
    print(type(s).__name__, "area =", s.area())
```

Output:- Rectangle area=15
circle area=153.94
Triangle area=17.0

Conclusion:-

Object oriented concepts were implemented in Python using classes and objects. Inheritance allowed child classes to reuse parent code, with super() linking the two. Polymorphism let one method name act differently across classes, keeping the code short and easy to extend.

Sundaram
FOR EDUCATIONAL USE